Name: Shreyansh Khaitan (CID: 06084579)
Submission date: 23 October 2026
Project: Can you still make money from triangular arbitrage in FX after real costs and real delays?

1. Hypothesis Pitch

The idea came from class. We learned about arbitrage recently and I got hooked on it. Honestly I always wanted to build an arbitrage model myself, and this capstone was the chance. Also as an international student in the UK I look at FX rates almost every day (getting money from home, paying in pounds etc), so I wanted to know if someone can actually make money from forex arbitrage, or if it only works on paper.

Triangular arbitrage is simple. Take EUR, GBP and USD. If the three rates don't line up, you can go EUR to GBP to USD and back to EUR and end up with more than you started. Most online examples, like Chau (2025), just multiply three rates and check if the answer is above 1. They don't include fees, the bid/ask spread or delay. I wanted to see what happens when you add all of that.

Why I think most of these chances are fake: every currency pair updates its price at slightly different times. So for a split second one leg of the loop is stale, and the loop looks profitable. But a real trader pays the spread on every leg, pays a fee, and can't trade instantly. By the time the order reaches the market, the stale price is already fixed. Fenn et al. found something similar, most deviations sit inside the spread and disappear within seconds. Ito et al. (2012) also show these chances got much shorter over time as trading became automated.

My hypothesis (H0): after paying the spread on every leg, paying fees, and filling 100 ms after the signal, the average profit per trade is not above zero out-of-sample.

What would prove me wrong: a positive average profit per trade out-of-sample with p-value under 5%, that still holds when I add more delay, higher fees, remove the best days and test other periods.

2. Data Pipeline

Dataset: Kaggle FX tick data (originally from Dukascopy), https://www.kaggle.com/datasets/zahinawosaf/forex-tick-data. 29 months (April 2020 to August 2022), 20 currency pairs. Each row is pair, timestamp in milliseconds, bid and ask. The notebook can download it with the Kaggle API, you just need your own token (no keys are in my submission).

Code: all my code is also on GitHub, https://github.com/shrey141102/Arbitrage-Project. The submitted notebook is the final version and all numbers in this report come from it.

The pipeline in simple steps:

1. Download. Get the monthly folders from Kaggle.
2. Snapshot. The full data is around 12 GB so I can't submit it. So I made a smaller snapshot with the notebook (Stage 2). For each month it first counts the ticks per day on EURUSD, skips weekends and any day with less than half the month's normal tick count (holidays, half days), and then picks 3 of the remaining days at random with a fixed seed (2026). So it's 87 days in total, and not hand picked. For those days

it keeps every single tick of the 15 pairs, no sampling inside the day (247 million ticks, about 1 GB), and saves them as compressed parquet files in data/snapshot/ticks/, one file per month and pair. It also writes a manifest.json with the chosen days, row counts, the source CSV for each file and a SHA-256 checksum, and the notebook checks every checksum before it runs. The snapshot is already built and included, so the notebook just loads it (REBUILD_SNAPSHOT = False) and you don't need the 12 GB. To rebuild it from scratch, put the Kaggle folders in data/kaggle/ and set REBUILD_SNAPSHOT = True. Because the seed is fixed it picks the same days again.

3. Clean. Six rules, and every removed row is counted: missing values, prices at or below zero, ask below bid, duplicate timestamps, spreads above 200 bps, and jumps above 100 bps from the median of the previous 51 ticks. 3.65% of ticks were removed, almost all were duplicate timestamps (9 million). The jump rule only looks backwards, because a real trader can't see the future.
4. Line up. Pairs tick at different times, so I made a 100 ms grid. At each grid time T every pair uses its last tick from strictly before T. I also save how old each quote is. This gives 36.3 million grid times.
5. Scan. At every grid time check every possible loop (next section).
6. Backtest. Trade, fill later, measure profit.

Missing data: holiday or half empty days are skipped when picking days. All 87 days were fine and no pair was missing on any day. If a pair was missing, its loops just wouldn't be checked, nothing is filled in. Splits and dividends don't exist in FX.

3. Strategy and Backtest

Model. Every currency is a node in a graph and every pair gives two edges. Selling the base currency uses the bid, buying it uses 1/ask. So the spread is already inside the graph. 5 pairs (EURPLN, USDMXN, USDRUB, USDTRY, USDZAR) don't form any loop with the others, so we end up with 15 pairs, 8 currencies and 90 loops (20 with 3 legs, 34 with 4 legs, 36 with 5 legs).

Formulas used:

- Edge rate: r = bid (selling base), r = 1/ask (buying base)
- Edge weight: w = -ln(r). A loop is profitable when the sum of w is below 0, which is the same as the product of r being above 1 (a negative cycle).
- Gross loop return (bps) = (r1 x r2 x … x rk - 1) x 10,000
- Net return after fees = (1 + gross)(1 - f)^k - 1, where f is the fee per leg and k the number of legs
- Daily return = sum of (net bps x $1M / 10,000) / $5M capital
- Cumulative return = sum of daily returns
- Sharpe = mean(daily return) / std(daily return) x √252
- Max drawdown = biggest (peak equity - equity) / peak equity
- Bootstrap p-value = share of resampled means that are 0 or below

Finding loops. A vectorised scanner checks all 90 loops at every grid time. I also wrote Bellman-Ford from scratch, like Akouaouch and Bouayad (2026) use. It finds a negative cycle of any length and rebuilds it by following parent pointers back. I checked both against each other.

Trading rule, at each time T:

1. Only look at loops where every quote is at most A ms old.
2. Pick the loop with the best profit after fees.
3. Trade only if that profit is above θ bps.
4. After a trade wait 1 second (so one mispricing isn't counted many times).
5. Fill the trade 100 ms later, at the prices at that later time.

Step 5 is the most important one. The signal only uses old info, and the result only uses prices from the fill time.

How I backtested. Fee 0.1 bps per leg, $1M per leg, $5M capital. θ and A were chosen from 25 combinations using the training months only, best training Sharpe won (θ = 1.0 bps, A = 2000 ms). The first 60% of months (Apr 2020 to Aug 2021, 51 days) were training. The last 12 months (Sep 2021 to Aug 2022, 36 days) were not touched until the settings were locked. I also did walk-forward: retune on all past months, test on the next 3 months, move forward, repeat (6 times, the last block only had 2 months left). For uncertainty I resampled whole days 2,000 times (day bootstrap), because trades on the same day are not independent.

Key results:

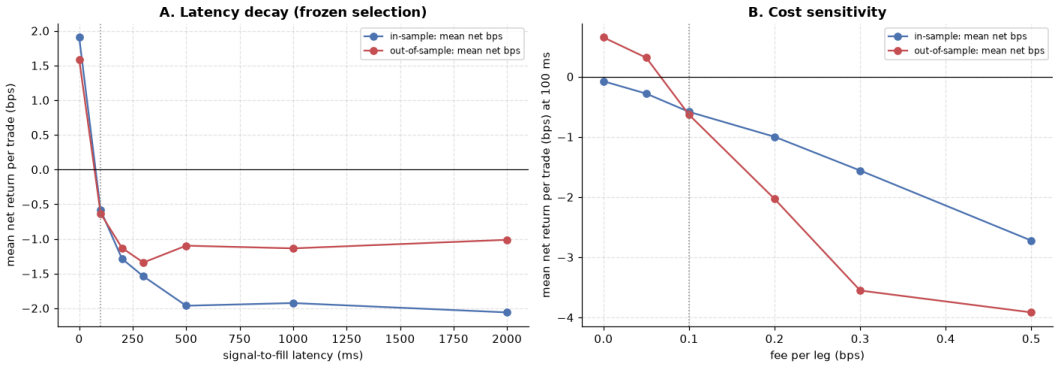
	In-sample (training)	**Out-of-sample (test)**
Trades	129	173
Mean profit per trade	-0.585 bps	-0.634 bps
95% CI	[-1.096, -0.232]	[-2.848, -0.020]
Hit rate	38.0%	54.3%
Cumulative return	-0.151%	-0.219%
Sharpe ratio	-6.30	-5.16
Max drawdown	0.151%	0.220%
p-value (H0: mean ≤ 0)	1.000	0.979

Walk-forward (all 6 test blocks together): 2,819 trades, mean +0.345 bps, 95% CI [-1.995, +0.413], p = 0.363, Sharpe +1.85. Only 1 of the 6 blocks was positive, and that was because of one single day (see pitfall checklist).

Note: Sharpe uses √252 but only on sampled days, so treat it as a rough number.

So H0 is not rejected. The strategy loses money. The main thing I found: on paper (fill at 0 ms) the test trades make +1.59 bps each. Fill them just 100 ms later and the same trades make -0.63 bps. So the profit was basically stale quotes.

The funnel also shows it well. 98.2% of grid times look profitable if you use mid prices. Only 5.5% still look profitable at real bid/ask. Only 302 trades pass all filters, and only 143 of them actually made money.



Tests. 14 checks are run inside the notebook and all pass. For example a hand made triangle with exactly +2.5 bps, a check of the fee formula, 800 random graphs where Bellman-Ford is compared with brute force, and the scanner checked against a slow plain Python version on 217,570 real values (difference 0). On 3,000 real grid times, Bellman-Ford and the scanner found the same 1,394 loops, 0 misses.

4. Methodology Defense

Why FX: it trades nearly 24 hours and is very liquid, so if arbitrage exists anywhere it’s a fair place to look. I used all 20 pairs in the dataset.

Why bid/ask and not mid: you can’t trade at the mid. With mids, 98% of the time looks like free money which is obviously wrong.

Why 100 ms delay: it’s one grid step, the smallest delay the data can show. It is very generous, real traders are slower. I tested 0 to 2000 ms as well.

Why the grid uses the right edge: if a 100 ms bin was labelled by its start, a decision at T could see ticks up to T + 100 ms, which is look-ahead. I built this wrong version on purpose as a test, and the test catches it.

Why 0.1 bps fee: roughly what a big bank or fund pays. For a normal person it’s much higher. I tested 0 to 0.5 bps.

Why a 1 second cool-down: the same mispricing often shows in several loops and several grid times. Without it I would count one chance many times.

Why split by month: so the test period is really in the future compared to training. Random days would mix future into the past.

Why 3 to 5 legs: triangles are the classic case, longer loops give more chances but pay more spread. I wanted to see if they help (they don’t).

What breaks if I’m wrong: longer delay or higher fees make results worse (see chart). Big orders would also move the price, but the Kaggle data has no volume so I can’t charge for that. So my results are, if anything, too optimistic.

5. Pitfall Checklist

Look-ahead bias

Checked in three ways. (1) At every grid time, the newest tick used is always older than the decision time, smallest gap was 1.0 ms. (2) I rebuild the data using only ticks up to a random cut-off and check nothing before the cut-off changes, passed at 40 out of 40 cut-

offs. (3) I made a version with a known look-ahead bug and the test caught it (23 of 40 cut-offs). Fill prices are always taken at the fill time.

Survivorship bias

Currencies don’t get delisted like stocks, so this is a smaller problem. All 15 pairs I used are there on all 87 days. But the dataset only has major pairs plus a few emerging ones, and the emerging ones drop out since they don’t close a loop. Less liquid pairs, where mispricings might be bigger, are not tested. So my result only covers the most liquid part of FX.

Overfitting/data-snooping

Only 2 settings (θ and A) were tuned, on 25 combinations, using training months only. In total 175 combinations were tried (25 at first, 150 during walk-forward). The hypothesis and test plan were fixed before seeing results.

Training results barely predict test results. Rank correlation across the 25 combinations is only +0.08, and my chosen one ranks 21 out of 25 on the test period. Some settings (θ = 0.25 to 0.5) look positive on the test period, but they all lost money in training, so choosing them now would be hindsight.

I also checked which days make the profit. Two days, 9 Feb 2022 and 28 Apr 2022, have most of the trades. In walk-forward, 28 Apr 2022 alone has 2,640 of 2,819 trades and makes +$116,045, which is more than the whole walk-forward profit (+$97,157). Without that day, walk-forward drops to -1.055 bps per trade (95% CI [-2.244, -0.337]). On the test period, 12 of 25 settings are positive, but only 6 stay positive without their best day, and those still have the other big day in them. 93% of trades winning after a 100 ms delay on a single day looks like a data problem, not a real edge.

Transaction costs and slippage

The spread is paid on every leg because the graph uses bid and ask. Fee is 0.1 bps per leg. The 100 ms delay works like slippage since prices move before the fill. With zero fee the test mean is +0.65 bps, at 0.05 bps it is +0.31 bps, at 0.1 bps it is already negative (-0.63 bps). So even a tiny fee kills it, and the zero fee result is probably also driven by the same two days. The data has no order book depth so big order costs aren’t modelled. As a small extra I ran one hour of Dukascopy data that does have volume (EUR, GBP, USD on 14 Oct 2026). 9 cases looked profitable at top of book, 0 after fees and a depth penalty.

Out-of-sample validation

Test period: Sep 2021 to Aug 2022 (36 days), held out until settings were locked. Training mean -0.585 bps, test mean -0.634 bps, so both periods agree it loses money. Walk-forward: only 1 of 6 blocks positive and that was one day. Without the best test day, the mean falls further to -1.093 bps.

6. Efficiency Touchpoint

The slowest step is the loop scan. It took about 70 seconds, around 60% of the run (whole notebook is about 2 minutes once the snapshot exists). Cost is O(S x C x k): S = 36.3 million grid times, C = 90 loops, k = 4.18 legs on average. That’s over 3 billion loop checks. Each delay level also needs a sorted as-of join, O(n log n).

Bellman-Ford is O(V x E), here V = 8 currencies and E = 30 edges, and it doesn't care how long the loop is. Per grid time: vectorised scanner about 0.4 µs for all 90 loops, plain Python loop about 18 to 19 µs, Bellman-Ford about 8.4 µs (these timings change a little on every run). Listing all loops is fine here because the graph is small, but the number of loops explodes with more currencies or longer loops, and then Bellman-Ford is the better choice. Memory is kept low by doing one day at a time in chunks.

7. Risk Touchpoint

Size: $1M per leg and about 3.4 legs per trade, so one trade is about 0.68x the capital and daily turnover is about 3.3x capital. Lots of trading for an edge under 1 bps.

Concentration: 32 loops traded, but the top one (EUR to JPY to USD to EUR) is 19.7% of trades and the top 5 are 67.6%. EUR is in 87% of trades and JPY in 86%. Effective number of loops (1 / sum of squared shares) is only about 8.9. So it's mostly a bet on EUR/JPY pricing.

Bad scenarios: if spreads double (liquidity crunch) mean goes to -2.54 bps, if they triple -4.46 bps. 500 ms delay gives -1.10 bps. Worst trade lost 20.4 bps, worst day lost $3,680.

Biggest real risk is leg risk (Kozhan and Tham, 2012). Two legs fill and the third doesn't, and now you're stuck holding a currency you didn't want. My backtest assumes all legs fill together so it doesn't show this.

8. Reflection

What I learned: I thought arbitrage was about finding loops where rates multiply to more than 1. Finding them is actually easy, trading them is the hard part. Almost all the profit was stale quotes and it was gone in 100 ms. I also learned how easy it is to cheat by mistake, just labelling a time bin wrong gives the strategy a look into the future.

What I got wrong:

- I thought fees would be the main problem. Delay was much worse. 100 ms alone turned +1.59 bps into -0.63 bps.
- I thought longer loops (4 or 5 legs) would help. They didn't, they pay more spread and lose more per trade after the delay.
- I trusted the walk-forward profit at first. Then I found it came from one day. Without checking day by day I would have reported a fake edge.
- In an earlier version of my code I had hardcoded numbers in some charts. I removed that and now everything is computed when the notebook runs.

Next steps: get data with order book depth to price bigger orders, use exchange timestamps to measure real delay, look at what happened on 9 Feb and 28 Apr 2022 (market event or bad data?), model leg risk, and test less liquid pairs. To really trade this you'd need a live feed, a faster language and servers next to the exchange. That's what HFT firms do, and it's probably why normal traders like me can't win this.

Disclaimer: use of AI

I used AI tools in this project. Gemini 4 Argon helped me brainstorm the idea and write parts of the code. Claude Opus 5.5 helped with the code, checking and fixing bugs (for

example the p-value fix and the best-day check), and writing this report. The idea, the choices and the final checks are mine, and I can explain every part of the pipeline.

References

- Code repository: https://github.com/shrey141102/Arbitrage-Project
- Dataset: Kaggle FX tick data (from Dukascopy), https://www.kaggle.com/datasets/zahinawosaf/forex-tick-data
- Akouaouch, I. and Bouayad, A. (2026). An innovative approach to identifying triangular arbitrage opportunities in financial markets using the Bellman-Ford algorithm. Bulletin of Electrical Engineering and Informatics, 15(3), 2579-2594. https://doi.org/10.11591/eei.v15i3.10817
- Chau, I. (2025). Triangular Arbitrage Explained (a naive algorithm), Part 1. Medium. https://medium.com/@ianchau379/triangular-arbitrage-explained-a-naive-algorithm-part-1-2e82d063ae4f
- Fenn, D., Howison, S., McDonald, M., Williams, S. and Johnson, N. The Mirage of Triangular Arbitrage in the Spot Foreign Exchange Market. arXiv:0812.0913. https://arxiv.org/abs/0812.0913
- Ito, T., Yamada, K., Takayasu, M. and Takayasu, H. (2012). Free Lunch! Arbitrage Opportunities in the Foreign Exchange Markets. NBER Working Paper 18541. https://www.nber.org/papers/w18541
- Kozhan, R. and Tham, W.W. (2012). Execution Risk in High-Frequency Arbitrage. Management Science, 58(11), 2131-2149.